

STUDENT ACTIVITY

Troubleshooting and Risk Mitigation — Activity

Activity: Identifying Neural Network Optimization Failures

Q: Read the four deep learning failure scenarios below. Identify which specific neural network component or training limitation is occurring in each situation: Batch Gradient Descent Inefficiency, Stochastic Gradient Descent Fluctuation, Missing Nonlinear Activation, or Failed Backpropagation.

Next, based on the provided text, describe why developers typically transition to Mini-Batch Gradient Descent when training Deep Learning models in three sentences. Ensure you explicitly mention how it balances performance, speed, and computational cost.

The Scenarios:

- A neural network struggles to learn highly complex relationships within image data because it acts as a simple linear transformation without introducing nonlinear learning.
- A deep learning model correctly calculates its loss function but continues to predict incorrect outputs over time because it fails to calculate the error at the output layer and propagate it backward to update the weights.
- A model's training process achieves high accuracy but is excessively slow for a large dataset because it attempts to use the complete dataset for every single weight update.
- A model learns faster but experiences erratic jumps and extreme fluctuations during its optimization process because it updates its weights using only one data point at a time.

Identifying Neural Network Optimization Failures

Scenario	Neural Network Component / Training Limitation	Explanation
1. A neural network struggles to learn highly complex relationships within image data because it acts as a simple linear transformation without introducing nonlinear learning.	Missing Nonlinear Activation	Without a nonlinear activation function (such as ReLU , Sigmoid , or Tanh), the network behaves like a linear model and cannot learn complex patterns in image data.
2. A deep learning model correctly calculates its loss function but continues to predict incorrect outputs because it fails to calculate	Failed Backpropagation	Backpropagation is responsible for calculating errors and updating weights. If it fails, the model cannot learn from its

Scenario	Neural Network Component / Training Limitation	Explanation
the error at the output layer and propagate it backward to update the weights.		mistakes, resulting in poor predictions.
3. A model's training process achieves high accuracy but is extremely slow for a large dataset because it uses the complete dataset for every weight update.	Batch Gradient Descent Inefficiency	Batch Gradient Descent processes the entire dataset before updating the weights, making training slow and computationally expensive for large datasets.
4. A model learns faster but experiences erratic jumps and extreme fluctuations because it updates its weights using only one data point at a time.	Stochastic Gradient Descent (SGD) Fluctuation	SGD updates weights after processing one training example at a time, leading to faster learning but unstable optimization with frequent fluctuations.

Why Developers Use Mini-Batch Gradient Descent

Developers use **Mini-Batch Gradient Descent** because it combines the advantages of **Batch Gradient Descent** and **Stochastic Gradient Descent**. It updates the model using **small batches of training data**, providing a good balance between **training performance, learning speed, and computational cost**. As a result, Mini-Batch Gradient Descent trains models more efficiently, reduces optimization fluctuations, and is the preferred method for most Deep Learning applications.

Identifying Neural Network Optimization Failures

Scenario	Training Limitation / Component	Reason
Neural network behaves like a linear model and cannot learn complex image patterns.	Missing Nonlinear Activation	Without nonlinear activation functions, the network cannot model complex relationships.
Loss is calculated but errors are not propagated to update weights.	Failed Backpropagation	Weights are not updated because error propagation fails.
Training is very slow because the full dataset is used for each weight update.	Batch Gradient Descent Inefficiency	Processing the entire dataset for every update increases computation time.

Scenario	Training Limitation / Component	Reason
Training is fast but optimization is unstable because weights are updated after each data point.	Stochastic Gradient Descent (SGD) Fluctuation	Single-sample updates cause noisy and fluctuating learning.

Mini-Batch Gradient Descent

Mini-Batch Gradient Descent updates the model using **small batches of training samples** instead of the entire dataset or a single sample. It provides an effective balance between **performance, speed, and computational cost**, making training faster and more stable. Therefore, it is the most commonly used optimization method for training modern Deep Learning models.